

SAA-C03 Personal Question Trainer Implementation Plan

Goal: 构建一个个人使用的 AWS SAA-C03 刷题网站，支持 PDF 题库导入、刷题、错题、笔记、模拟考试、跨设备云同步和手机使用。

Architecture: 采用前端优先架构：题库本身作为静态 JSON 随网站部署，用户刷题进度先保存在本地 IndexedDB，再同步到 Supabase。第一版避免复杂后端，把后端职责限制在登录、用户进度同步和数据持久化。

Tech Stack: Vite + React + TypeScript + Tailwind CSS + Dexie/IndexedDB + Supabase Auth/Postgres + Python PDF parser + Vitest + Playwright + Vercel/Cloudflare Pages。

1. 产品边界

这个项目不是公开 SaaS 刷题平台，而是你的个人刷题工具。

MVP 要解决：

- 从 `AWS SAA-C03 英文 1019Q.pdf` 解析出题库
- 网页刷题
- 手机和电脑都能用
- 登录后跨设备同步进度
- 错题、蒙对题、收藏题、笔记
- 65 题模拟考试
- 基础统计

MVP 不做：

- 付费系统
- 管理后台
- 多题库市场
- 排行榜
- 公开用户社区
- AI 自动讲解

- 复杂 spaced repetition 算法
- 多租户组织权限

2. 最终技术栈选择

Frontend

```
Vite
React
TypeScript
Tailwind CSS
lucide-react
```

理由：

- 这是交互型个人学习工具，不需要 SSR。
- Vite 静态部署简单，适合 Vercel、Netlify、Cloudflare Pages。
- React 适合处理刷题状态、筛选、计时器、统计面板。
- TypeScript 能约束题目、答案、进度、考试记录这些结构化数据。

Local Storage

```
Dexie
IndexedDB
```

理由：

- 比 `localStorage` 更适合保存大量结构化刷题记录。
- 支持离线使用。
- 后续可以做“本地先写入，再同步云端”。

Cloud Sync

```
Supabase Auth
Supabase Postgres
```

理由：

- 你只需要个人登录和进度同步，不需要自己写完整后端。
- Postgres 很适合保存用户答题状态、考试记录、笔记。
- Supabase Auth 可以快速支持邮箱登录或 magic link。
- RLS 可以保证每个用户只能看到自己的数据。

Data Import

```
Python  
pypdf / pdfplumber
```

理由：

- PDF 解析是离线构建步骤，不应该放进浏览器。
- Python 处理 PDF、正则清洗、生成报告更直接。
- 前端只消费结构化后的 `questions.json`。

Testing

```
Vitest  
React Testing Library  
Playwright
```

理由：

- Vitest 测判题、统计、筛选、同步合并逻辑。
- Playwright 测真实刷题流程和考试流程。
- 不过度测试 UI 样式，重点测会影响学习结果的逻辑。

3. 系统架构

PDF 题库

↓

Python parser

↓

questions.json

↓

Vite React App

↓

浏览器刷题 UI

用户状态:

React UI

↓

Dexie / IndexedDB

↓

Sync Service

↓

Supabase Auth + Postgres

关键原则:

- 题库是静态资源。
- 用户数据是动态资源。
- 本地 IndexedDB 是第一写入点。
- Supabase 是跨设备同步层。
- App 即使暂时离线, 也应该能继续刷题。

4. 推荐项目结构

```
saa-c03-trainer/  
  package.json  
  vite.config.ts  
  tsconfig.json  
  index.html  
  .env.example  
  
  scripts/
```

```
parse_pdf.py
validate_questions.py
```

```
data/
  raw/
    AWS SAA-C03 英文 1019Q.pdf
  questions.json
  parse-report.json
```

```
supabase/
  migrations/
    001_initial_schema.sql
  seed.sql
```

```
src/
  main.tsx
  App.tsx
  styles.css
```

```
data/
  questions.json
  questions.ts
```

```
domain/
  question.ts
  progress.ts
  session.ts
  stats.ts
  exam.ts
  sync.ts
```

```
db/
  localDb.ts
  progressRepository.ts
  examRepository.ts
```

```
cloud/
  supabaseClient.ts
  authService.ts
  syncService.ts
```

```
components/
  AppShell.tsx
  Dashboard.tsx
```

```
QuestionCard.tsx
AnswerOptions.tsx
ExplanationPanel.tsx
QuestionNavigator.tsx
ModeSwitcher.tsx
ReviewControls.tsx
ExamTimer.tsx
SyncStatus.tsx
```

pages/

```
DashboardPage.tsx
PracticePage.tsx
ReviewPage.tsx
ExamPage.tsx
SettingsPage.tsx
LoginPage.tsx
```

tests/

```
progress.test.ts
stats.test.ts
exam.test.ts
sync.test.ts
```

e2e/

```
practice.spec.ts
exam.spec.ts
sync.spec.ts
```

5. 数据模型

Question

```
export type ChoiceKey = "A" | "B" | "C" | "D" | "E" | "F";

export type Question = {
  id: number;
  stem: string;
  options: Partial<Record<ChoiceKey, string>>;
  answer: ChoiceKey | ChoiceKey[];
```

```
  explanation: string;
  sourcePage?: number;
};
```

QuestionProgress

```
export type QuestionProgress = {
  questionId: number;
  attempts: number;
  correctAttempts: number;
  lastSelected: ChoiceKey[];
  lastResult?: "correct" | "incorrect";
  markedGuessed: boolean;
  bookmarked: boolean;
  note: string;
  updatedAt: string;
  syncedAt?: string;
};
```

ExamSession

```
export type ExamSession = {
  id: string;
  questionIds: number[];
  startedAt: string;
  submittedAt?: string;
  durationSeconds: number;
  answers: Record<number, ChoiceKey[]>;
  score?: number;
};
```

6. Supabase Schema

question_progress

```
create table question_progress (  
  user_id uuid not null references auth.users(id) on delete cascade,  
  question_id integer not null,  
  attempts integer not null default 0,  
  correct_attempts integer not null default 0,  
  last_selected text[] not null default '{}',  
  last_result text check (last_result in ('correct', 'incorrect')),  
  marked_guessed boolean not null default false,  
  bookmarked boolean not null default false,  
  note text not null default '',  
  updated_at timestamptz not null default now(),  
  primary key (user_id, question_id)  
);
```

```
alter table question_progress enable row level security;
```

```
create policy "Users can read own progress"  
on question_progress for select  
using (auth.uid() = user_id);
```

```
create policy "Users can insert own progress"  
on question_progress for insert  
with check (auth.uid() = user_id);
```

```
create policy "Users can update own progress"  
on question_progress for update  
using (auth.uid() = user_id);
```

exam_sessions

```
create table exam_sessions (  
  id uuid primary key default gen_random_uuid(),  
  user_id uuid not null references auth.users(id) on delete cascade,  
  question_ids integer[] not null,  
  answers jsonb not null default '{}',  
  started_at timestamptz not null default now(),  
  submitted_at timestamptz,  
  duration_seconds integer not null default 0,  
  score integer  
);
```

```
alter table exam_sessions enable row level security;

create policy "Users can manage own exam sessions"
on exam_sessions for all
using (auth.uid() = user_id)
with check (auth.uid() = user_id);
```

7. 开发任务拆分

Task 1: 初始化前端项目

Files:

- Create: `package.json`
- Create: `vite.config.ts`
- Create: `tsconfig.json`
- Create: `src/main.tsx`
- Create: `src/App.tsx`
- Create: `src/styles.css`

Steps:

- 创建 Vite React TypeScript 项目。
- 安装 Tailwind CSS。
- 安装 `lucide-react`、`dexie`、`@supabase/supabase-js`。
- 安装测试依赖：`vitest`、`@testing-library/react`、`playwright`。
- 确认 `npm run build` 成功。

Task 2: PDF 解析脚本

Files:

- Create: `scripts/parse_pdf.py`
- Create: `scripts/validate_questions.py`
- Create: `data/questions.json`

- Create: `data/parse-report.json`

Steps:

- 用 Python 读取 PDF 文本。
- 按题号切分题目。
- 提取题干、选项、答案、解析。
- 输出 `questions.json`。
- 输出解析报告，记录缺答案、缺选项、缺解析的题号。
- validator 检查题目数量、重复 ID、字段完整性。
- 人工抽查 Q1-Q10、Q100-Q110、Q500-Q510、Q1000-Q1019。

成功标准：

Validated 1019 questions

Task 3: 题目与判题领域逻辑

Files:

- Create: `src/domain/question.ts`
- Create: `src/domain/progress.ts`
- Create: `src/tests/progress.test.ts`

Core functions:

```
gradeAnswer(correctAnswer, selectedAnswer)
normalizeAnswer(answer)
updateProgressAfterAnswer(progress, selected, result)
```

Tests:

- 单选正确。
- 单选错误。
- 多选正确，顺序无关。
- 多选漏选算错。

- 提交后 attempts 增加。
 - 答对后 correctAttempts 增加。
-

Task 4: 本地数据库 Dexie

Files:

- Create: `src/db/localDb.ts`
- Create: `src/db/progressRepository.ts`
- Create: `src/db/examRepository.ts`

Tables:

```
progress: "questionId, updatedAt, syncedAt, lastResult, markedGuessed, bookmarked"
examSessions: "id, startedAt, submittedAt"
```

Steps:

- 定义 Dexie database。
 - 实现保存单题进度。
 - 实现读取全部进度。
 - 实现按题号读取进度。
 - 实现保存考试记录。
 - 实现清空本地数据。
-

Task 5: 刷题页面

Files:

- Create: `src/pages/PracticePage.tsx`
- Create: `src/components/QuestionCard.tsx`
- Create: `src/components/AnswerOptions.tsx`
- Create: `src/components/ExplanationPanel.tsx`

- Create: `src/components/QuestionNavigator.tsx`

Behavior:

- 显示当前题目。
 - 提交前隐藏答案和解析。
 - 支持单选和多选。
 - 提交后显示正确/错误。
 - 提交后显示解析。
 - 保存答题结果到 IndexedDB。
 - 支持上一题/下一题。
-

Task 6: 刷题模式

Files:

- Create: `src/domain/session.ts`
- Create: `src/components/ModeSwitcher.tsx`

Modes:

```
type PracticeMode =  
  | "sequential"  
  | "random"  
  | "wrong"  
  | "guessed"  
  | "bookmarked";
```

Steps:

- 顺序刷题。
- 随机刷题。
- 错题模式。
- 蒙对题模式。
- 收藏题模式。
- 对空结果显示 empty state。

Task 7: 笔记、收藏、蒙对标记

Files:

- Create: `src/components/ReviewControls.tsx`
- Modify: `src/pages/PracticePage.tsx`

Behavior:

- 当前题可标记收藏。
 - 当前题可标记蒙对。
 - 当前题可写个人笔记。
 - 笔记自动保存。
 - 刷新页面后标记和笔记仍然存在。
-

Task 8: Dashboard 统计

Files:

- Create: `src/domain/stats.ts`
- Create: `src/pages/DashboardPage.tsx`
- Create: `src/components/Dashboard.tsx`
- Create: `src/tests/stats.test.ts`

Stats:

- 总题数
- 已刷题数
- 剩余题数
- 正确率
- 错题数
- 蒙对题数
- 收藏题数

Tests:

- 没有答题时正确率是 0。

- 2 题做完 1 对 1 错，正确率是 50。
 - 收藏和蒙对统计正确。
-

Task 9: 模拟考试模式

Files:

- Create: `src/domain/exam.ts`
- Create: `src/pages/ExamPage.tsx`
- Create: `src/components/ExamTimer.tsx`
- Create: `src/tests/exam.test.ts`

Behavior:

- 随机抽 65 道不重复题。
 - 130 分钟计时。
 - 考试中可前后切题。
 - 提交后显示分数。
 - 未答题算错。
 - 提交后可查看错题和解析。
 - 考试记录保存本地。
-

Task 10: Supabase 登录

Files:

- Create: `src/cloud/supabaseClient.ts`
- Create: `src/cloud/authService.ts`
- Create: `src/pages/LoginPage.tsx`
- Create: `.env.example`

Environment:

```
VITE_SUPABASE_URL=  
VITE_SUPABASE_ANON_KEY=
```

Behavior:

- 支持邮箱 magic link 或 email/password。
 - 登录状态持久化。
 - 未登录也可以本地刷题。
 - 登录后提示是否同步本地数据。
-

Task 11: 云同步

Files:

- Create: `src/cloud/syncService.ts`
- Create: `src/domain/sync.ts`
- Create: `src/components/SyncStatus.tsx`
- Create: `src/tests/sync.test.ts`

Sync Strategy:

- 本地是第一写入点。
- 每次答题先写 IndexedDB。
- 登录后后台同步到 Supabase。
- 冲突时使用 `updatedAt` 较新的记录。
- 如果本地和云端都有同一题进度，合并字段时保留更高价值数据：
 - `attempts` 取较大值或累加，MVP 可取较大值。
 - `correctAttempts` 取较大值。
 - `bookmarked` 只要有一边为 true 就保留 true。
 - `markedGuessed` 只要有一边为 true 就保留 true。
 - `note` 取更新时间较新的版本。

Tests:

- 本地新记录能上传。
- 云端新记录能下载。
- 本地更新较新时覆盖云端。
- 云端更新较新时覆盖本地。

- 收藏标记合并不丢失。
-

Task 12: 手机体验与 PWA

Files:

- Modify: `src/styles.css`
- Modify: `src/components/AppShell.tsx`
- Create: `public/manifest.webmanifest`

Behavior:

- 手机单列布局。
 - 选项按钮足够大，适合触摸。
 - 模拟考试计时器固定可见。
 - 支持添加到主屏幕。
 - 弱网时仍可刷已加载题库。
 - 同步失败时显示非阻塞提示。
-

Task 13: 端到端测试

Files:

- Create: `playwright.config.ts`
- Create: `e2e/practice.spec.ts`
- Create: `e2e/exam.spec.ts`

E2E flows:

- 打开首页，看到总题数。
 - 进入刷题页，选择答案，提交，看到解析。
 - 标记收藏，切到收藏模式，题目存在。
 - 开始模拟考试，看到 65 题和计时器。
 - 提交考试，看到分数。
-

Task 14: 部署

Option A: Vercel

```
Build command: npm run build
Output directory: dist
```

Option B: Cloudflare Pages

```
Build command: npm run build
Output directory: dist
```

Environment variables:

```
VITE_SUPABASE_URL
VITE_SUPABASE_ANON_KEY
```

Checks:

- 桌面浏览器可访问。
- 手机浏览器可访问。
- 登录成功。
- 手机答题后电脑能同步看到。
- 电脑收藏题后手机能同步看到。

8. 开发顺序建议

最稳的执行顺序：

1. PDF parser + validator
2. 静态题库加载
3. 判题逻辑和测试
4. Dexie 本地进度

5. 基础刷题页
6. 错题/蒙对/收藏/笔记
7. Dashboard
8. 模拟考试
9. Supabase Auth
10. Supabase progress sync
11. 手机适配和 PWA
12. Playwright e2e
13. 云端部署

这个顺序的重点是：先把本地刷题闭环做通，再加云同步。这样不会一开始就被登录、权限、同步冲突拖慢。

9. MVP 验收标准

MVP 完成时，你应该能做到：

- 在电脑打开网站刷题。
 - 在手机打开同一个网站刷题。
 - 登录同一个账号后进度同步。
 - 能看到自己做过多少题。
 - 能只刷错题。
 - 能只刷蒙对题。
 - 能收藏题。
 - 能给题写笔记。
 - 能做 65 题模拟考试。
 - 刷新页面不丢记录。
 - 网络短暂断开时，本地进度不丢。
-

10. 关键工程风险

PDF 解析风险最高。

题库 PDF 可能有换行、页眉页脚、选项断行、解析跨页等问题。要先做 parser report 和人工抽查。

同步冲突要保持简单。

MVP 不做复杂 CRDT。用 `updatedAt` 解决大多数冲突即可。

题库不要放 Supabase。

1019 道题是静态内容，直接随前端部署为 JSON 更简单。Supabase 只保存你的个人进度。

不要过早做 AI。

AI 讲解很诱人，但第一版最重要的是刷题效率。先把错题闭环做好。

11. 最终推荐版本

我推荐你做：

```
Vite + React + TypeScript  
Tailwind CSS  
questions.json  
Dexie + IndexedDB  
Supabase Auth + Postgres  
Python PDF parser  
Vercel or Cloudflare Pages
```

这套方案刚好覆盖你的真实需求：**个人刷题、本地快、云端可访问、跨设备同步、手机能用**。它不会像纯本地版那样被设备限制，也不会像完整 SaaS 那样过度复杂。